

# Praktikum Pemrograman Berbasis Objek

Pertemuan 1

# Asisten Praktikum PBO 2026



Rosa Andini Ismayanti

## Kelas A



Muhammad Zefa Inzaghi



Bagas Anung Bayu Nugraha

## Kelas B



Fardan Fadhilah Andicha Putra



Haris Herdiansyah

## Kelas C



Sammy Farrel Zebua

# Rules Praktikum

1. Praktikan wajib mengikuti praktikum sesuai jadwal kelas masing-masing yaitu:

- Kelas A : Hari Selasa pukul 10.30 - 12.30 WIB
- Kelas B : Hari Senin pukul 10.30 - 12.30 WIB
- Kelas C : Hari Rabu pukul 08.00 - 10.00 WIB

Apabila ada yang bentrok dengan jadwal yang lain harap menghubungi asprak.

2. Praktikan yang tidak mengikuti praktikum lebih dari **3 kali** tidak diperkenankan untuk mengikuti UAS.
3. Platform yang digunakan untuk praktikum PBO adalah Discord dan Github Classroom.
4. Praktikan diharap dapat mengikuti praktikum dengan baik, tidak menyontek, dan menjaga kerja sama yang sehat.
5. Untuk peraturan mendatang khusus mengenai Discord dan Github Classroom dapat dipantau di channel announcement Discord Praktikum PBO 2026.

# Penilaian

Nilai yang diambil adalah:

- Kehadiran
- Tugas
- Evaluasi
- UAS/Projek

# Apa yang akan dipelajari?

01

## Konsep PBO

Memahami konsep PBO terutama dalam bahasa Java atau bahasa lainnya.

02

## Pemrograman dalam Java

Memahami struktur kode dan penerapan PBO dalam Java.

03

## Bahasa Pemrograman Java

Mengenal berbagai macam class dan framework dalam Java.

04

## Membuat Program Java

Menerapkan konsep PBO dengan membangun program Java.

# Materi Pertemuan 1

Gambaran Materi Pembelajaran Hari Ini

# Materi Pertemuan 1

1. Konsep PBO
2. Struktur Kode Java
3. Membuat Objek dari Class
4. Input & Output
5. Statements dalam Java
6. Compile & Run Kode Java

Lanjut →

# Konsep PBO

Gambaran Umum Mengenai Dasar dan Konsep PBO



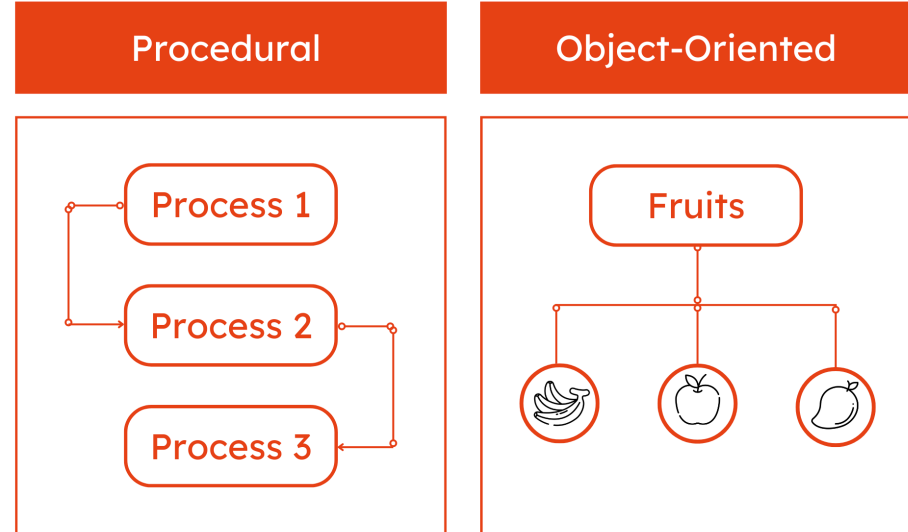
# Prosedural vs PBO

## Prosedural

- Program dalam bentuk instruksi linier (step-by-step).
- Data dan fungsi terpisah; fungsi memanipulasi data global/lokal.
- Sulit dikelola saat basis kode menjadi besar (complex & tightly coupled).

## Pemrograman Berorientasi Objek

- Program diorganisasi menjadi entitas-entitas mandiri bernama Objek.
- Data dan operasi yang memanipulasinya disatukan ke dalam satu kesatuan.
- Lebih modular, terstruktur, dan mudah dikembangkan (scalable).



Setiap entitas di dunia nyata dapat direpresentasikan menjadi objek di dalam kode:

1. *State (Atribut / Variabel / Data):*

- Karakteristik atau informasi yang dimiliki objek.
- Contoh (Mahasiswa): `nama` , `npm` , `ipk` .

2. *Behavior (Method / Fungsi / Aksi):*

- Tindakan atau operasi yang dapat dilakukan oleh objek.
- Contoh (Mahasiswa): `belajar()` , `isiKRS()` , `cetakNilai()` .

# Keunggulan PBO

- **Modularity**

Kode terbagi rapi ke dalam komponen-komponen mandiri.

- **Reusability**

Class yang sudah dibuat dapat digunakan kembali tanpa menulis ulang logika dari nol.

- **Maintainability**

Perubahan pada satu bagian sistem tidak mudah merusak bagian lainnya.

- **Real-world intuition**

Struktur kode merefleksikan cara kerja sistem di dunia nyata (sistem perbankan, game, dll).

# Struktur Code Java

Overview Mengenai Struktur Code Java

# Struktur Code Java (1)

```
public class HelloWorld {  
    public static void main(String args[]){  
        System.out.println("Hello World from Java!");  
    }  
}
```

## Class

- `public` : Access Modifier
- `class` : Class keyword
- `HelloWorld` : Name of Class

## Method

- `static` : Static keyword
- `void` : Return type
- `main` : Method
- `String args[]` : Parameter.

`String args[]` as default parameter for method  
`main`

# Struktur Code Java (2)

```
public class Ship {  
    // Variable  
    private int crew;  
  
    // Constructor: Method yang dilakukan saat instansiasi class menjadi object  
    public Ship(){  
        this.crew = 24;  
    }  
  
    // Setter method: Method untuk assign variable dalam class  
    public void setCrew(int crew){  
        this.crew = crew;  
    }  
  
    // Getter method: Method untuk mengambil nilai variable suatu class  
    public int getCrew(){  
        return this.crew;  
    }  
}
```

# Membuat Objek dari Class

Class Baru dapat Digunakan Apabila di Instansiasikan Menjadi Objek

# Membuat Object dari Class

```
public class Test {  
    public static void main(String args[]){  
        Ship ferry = new Ship();  
  
        if (ferry.getCrew() < 30){  
            ferry.setCrew(30);  
        }  
  
        System.out.println("The following ship has " + ferry.getCrew() + " crew.");  
    }  
}
```

Instansiasi class menjadi object dilakukan dengan keyword `new` diikuti dengan class yang akan dijadikan object



# Static Method

```
public class Ship {  
    private String name;  
  
    public Ship(String name) {  
        this.name = name;  
    }  
  
    // 1. Instance Method (Perlu objek: ferry.sail())  
    public void sail() {  
        System.out.println("Kapal " + this.name + " berlayar.");  
    }  
  
    // 2. Static Method (Langsung via Class: Ship.info())  
    public static void info() {  
        System.out.println("Semua kapal berlayar di laut.");  
    }  
}
```

Static method terikat langsung pada *Class*, bukan pada objek individual. Kita tidak perlu membuat `new Ship()` untuk memanggil `info()`.

# Static Method

```
public class Main {  
    public static void main(String[] args) {  
        Ship.info();           // 1. Static Method (Langsung via Class, tanpa 'new')  
        Ship ferry = new Ship("Merak");  
        ferry.sail();          // 2. Instance Method (Dipanggil dari objek 'ferry')  
    }  
}
```

| Aspek         | Instance Method                                               | Static Method                                                 |
|---------------|---------------------------------------------------------------|---------------------------------------------------------------|
| Kepemilikan   | Milik masing-masing objek                                     | Milik <code>Class</code> secara keseluruhan                   |
| Pemanggilan   | <code>objek.method()</code> ( <i>butuh <code>new</code></i> ) | <code>Class.method()</code> ( <i>tanpa <code>new</code></i> ) |
| Akses Konteks | Mengenali keyword <code>this</code> / <code>self</code>       | Tidak mengenal <code>this</code> / <code>self</code>          |
| Fungsi Utama  | Mengolah data/status unik objek                               | Operasi <i>helper, utility</i> , & fungsi umum                |

# Input & Output

Operasi Input dan Output melalui CLI dalam Java

# Input & Output

```
// import class Scanner dari package java.util
import java.util.Scanner;

public class Test {
    public static void main(String args[]){
        // Instansiasi objek scanner
        Scanner sc = new Scanner(System.in);

        //Input dengan objek scanner
        String name = sc.nextLine();
        int age = Integer.parseInt(sc.nextLine());

        // Output dengan System.out
        System.out.println("Namaku adalah " + name);
        System.out.println("Umurku " + age + " tahun.");

        // Menutup objek scanner supaya tidak terjadi memory leak
        sc.close();
    }
}
```

# Statement dalam Java

Overview mengenai If, Switch, and Looping Statements dalam Bahasa Java

# Statement dalam Java

Contoh if dan loop statement :

```
public class Loop{  
    public static void main(String args[]){  
        for(int i = 1; i ≤ 10; i++){  
            if(i > 7){  
                break;  
            }  
            System.out.println("Loop no. " + i);  
        }  
    }  
}
```

Statement if, else, for, while, do-while, dan switch dalam bahasa pemrograman Java memiliki cara penulisan yang sama dengan bahasa pemrograman C++.

Berikut adalah contoh looping dan if for saat menggunakan Java, silahkan coba-coba statement lainnya.

# Compile & Run

Bagaimana Compile dan Menjalankan File Java yang Sudah Dibuat

# Compile and Run

## - Compile File Java

Untuk compile file java run command berikut di terminal:

```
javac FileName.java
```

Command diatas akan menghasilkan file baru dengan nama `FileName.java`

## - Run File Java

Untuk menjalankan file hasil compile run command berikut di terminal:

```
java FileName
```

Command diatas akan menjalankan hasil compile tanpa membawa type class-nya



# Important Things About Compile and Running Java

- Nama file `.java` harus sama dengan nama class-nya.

contoh : HelloWorld.java harus memiliki class bernama HelloWorld

- Dalam satu file `.java` dapat memiliki banyak class namun hanya diperbolehkan memiliki **satu** public class
- Disarankan untuk class yang bersifat public untuk memiliki filenya sendiri-sendiri

# Assignment!

Seperti biasa setiap selesai praktikum pasti akan ada tugas

# Assignment

## CLI Menu Interaktif

Buatlah program CLI berbasis menu interaktif dengan ketentuan berikut:

### 1. Class Entitas

Misal: Produk / Kendaraan / Buku .

Wajib memiliki minimal 2 instance attributes dan 1 static attribute (*counter objek*).

### 2. Skema Menu Perulangan

- Input Data Baru: Menerima input & buat objek baru.
- Cek Total Data: Panggil static method total objek.
- Keluar: Menghentikan program.

CONTOH OUTPUT:

```
≡≡≡ SISTEM MANAJEMEN BUKU ≡≡≡  
1. Input Data Baru  
2. Cek Total Data  
3. Keluar  
Pilih menu (1-3):
```

# Teknis Pengumpulan

Teknis pengumpulan tugas akan diinformasikan dikemudian hari.

# Deadline Pengumpulan

Kelas A:

07 September 2026

Kelas B:

30 Agustus 2026

Kelas C:

01 September 2026

Waktu yang dilihat adalah waktu last commit. Jika ada yang commit melewati deadline walaupun sudah commit sebelumnya akan dianggap telat

# Terima Kasih!

Do you have any questions? Please use respective class discussion channel on Discord.

Semangat terus menjalani kuilahnya!! 🔥🔥🔥